

# RAPPORT EVALBOT:

## Eval-Pong

Réalisé par : Abdelmoumen Bourzami , Zakaria Boschetti E3S (ingé1) Esiee Paris

Groupe : 1s1



Supervisé par : IGI-3001 – Architecture : Programmation et systèmes à base  
microprocesseur – M. Kachouri

## INTRODUCTION:

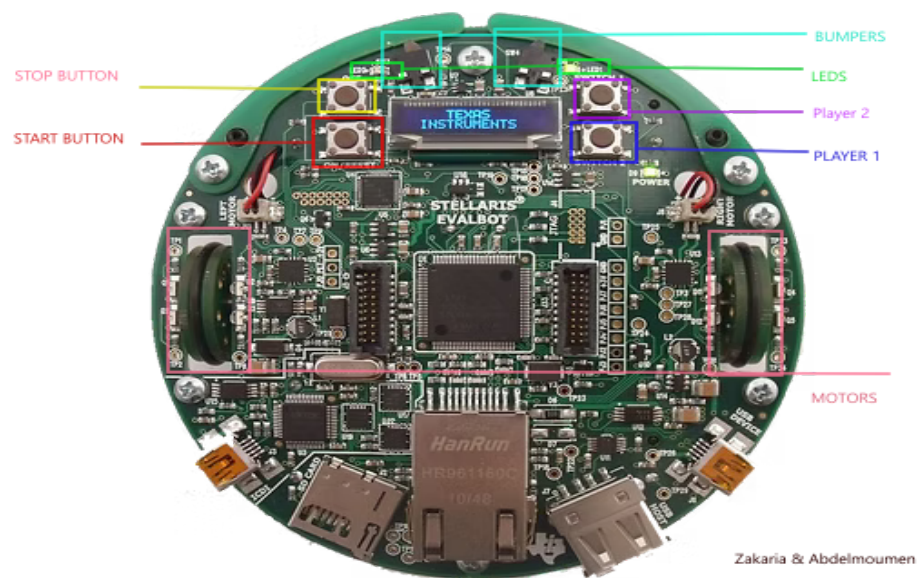
Afin de réaliser notre projet nous nous sommes inspirés du premier jeu vidéo : PONG . Nous avons ainsi considéré le robot comme étant la balle mais également le système de compte . L'ensemble du code est écrit entièrement en assembleur, afin de maîtriser la configuration des registres matériels, des GPIO et du PWM.

## PRINCIPE :

Tout comme le pong le robot evalbot est censé démarrer dans une direction aléatoire , de rebondir contre les parois mais également de compter le score et accéléré à chaque rebond sur des parois .

## MATÉRIEL & Définition :

1. Evalbot (robot) + trois piles
2. Arène



Définition :

## PROCÉDURE :

1. Allumer l'évalbot ce dernier démarre alors et effectue une rotation aléatoire afin de lancer le jeu et de démarrer dans une direction aléatoire
2. Lors de la rencontre avec un obstacle comme un mur , les cages d' un des joueur ou tout autre corps étranger présent dans l'arène un ou les deux Bumpers du robot ce dernier effectue une rotation dans le sens horaire ou anti horaire en fonction du bumper touché (respectivement droite ou gauche ) de façon aléatoire avant de continuer sa course
3. Lorsque le robot entre dans les cages du joueur, celui-ci est remplacé au centre de l'arène par le joueur et le un point est ajouté au joueur adverse en utilisant le bouton 1 ( pour le joueur 1) et le bouton 2 (pour le joueur 2) pour un maximum de 5 points par joueur.
4. Le score étant alors incrémenté celui-ci sera affiché tout au long de la partie au travers des leds (led1 pour le joueur 1 et led2 pour le joueur 2) une fois qu' un des joueur atteint 5 points la partie est considérée comme finie et les moteurs se coupent .
5. Le jeu est alors relancé en appuyant sur le Bouton Start et les scores sont remis à zéros .

## EXPLICATIONS SIMPLIFIÉE :

Le projet repose sur deux modules indépendants mais complémentaires :

- Le module de pilotage moteur et gestion des bumpers
- Le module de gestion des LEDs et du score

### MODULE DE PILOTAGE :

Ce module est le cœur du déplacement du robot.

Il configure les périphériques GPIO, le timer SysTick et les moteurs pour permettre un comportement **réactif et autonome**.

### Configuration matérielle

- **Ports utilisés :**
  - GPIO E : bumpers gauche (E0) et droit (E1)
  - GPIO F : LEDs F4 et F5
- Activation des **horloges GPIOE et GPIOF**.
- Mise en place du **timer SysTick** comme source de délais et d'aléa.
- Appel aux sous-programmes d'initialisation des moteurs :

MOTEUR\_INIT, MOTEUR\_GAUCHE\_ON, MOTEUR\_DROIT\_ON.

|     |    |                     |
|-----|----|---------------------|
| 121 | BL | MOTEUR_GAUCHE_AVANT |
| 122 | BL | MOTEUR_DROIT_AVANT  |

### Boucle principale

Le programme principal exécute une boucle infinie :

1. Les deux moteurs sont activés pour avancer.
2. L'état des bumpers est lu en permanence.
3. Si un bumper est pressé, le robot réagit selon trois cas :
  - **Bumper gauche** : recule légèrement et tourne à droite.
  - **Bumper droit** : recule légèrement et tourne à gauche.

- **Les deux bumpers** : le robot exécute un virage à 200°.
4. Après chaque choc, une temporisation de sécurité évite les doubles détections.

### Comportement aléatoire

Une fonction **RANDOM\_DELAY** introduit un temps de rotation pseudo-aléatoire en combinant :

- la valeur du compteur **SysTick**(Timers & pseudo-aléatoire),

```

265          LDR      r0, =0xE000E018
266          LDR      r1, [r0]
```

- et l'état instantané des **entrées des bumpers**.  
Cela rend les mouvements du robot moins répétitifs et plus naturels.

### Résultat observé

Le robot se déplace de manière autonome, détecte les obstacles et adapte sa direction en toute autonomie.

Les mouvements sont fluides, les rotations aléatoires, et les collisions bien gérées.

## EXPLICATIONS DÉTAILLÉE ET JUSTIFICATION :

### 1. Choix et justification des ports GPIO

#### **A)Moteurs et Ports**

##### **Port D**

Le **port D** est utilisé pour le contrôle des deux ponts en H (DRV8801) assurant l'alimentation des moteurs à courant continu.

Les broches **PD0** et **PD1** délivrent respectivement les signaux **PWM0** et **PWM1**, directement reliés aux entrées de commande du premier moteur.

Les broches **PD2** et **PD5** servent quant à elles au contrôle du mode de décharge lente (“Slow Decay”) et à l’activation de l’alimentation 12 V.

Ce choix s’explique par le **câblage natif de l’Evalbot**, où le port D est physiquement interfacé avec les entrées de puissance du DRV8801.

L’utilisation de ces broches permet donc de générer un signal Pulse With Modulation (PDW) **matériellement stable**, sans surcharge logicielle, et garantit une **synchronisation parfaite** entre le microcontrôleur et les ponts en H comme indiqué par les divers intervenant .

### **Port H – Commande du second moteur**

Le **port H** est affecté au pilotage du second moteur via les signaux **PWM2** et **PWM3** (sorties **PH0** et **PH1**).

Ce choix assure une **symétrie matérielle** entre les deux canaux de commande, chacun disposant de son propre générateur PWM matériel.

Il en résulte une indépendance totale entre les deux voies, ce qui facilite la mise en œuvre de trajectoires différentielles (rotation, virage, alignement).

Le port H a été retenu pour sa **proximité logique avec le module PWM1**, ce qui limite la latence d’accès et garantit une modulation temporelle de haute précision.

### ***B) Collisions***

#### **Port E – Détection des bumpers**

Le **port E** est réservé à la lecture des capteurs de contact (bumpers) situés à l’avant du robot, sur les broches **PE0** et **PE1**.

### ***C) LEDs***

#### **Port F – Indicateurs visuels (LEDs)**

Le **port F** est utilisé pour la gestion des **LEDs PF4 et PF5**, intégrées à la carte Evalbot.

Ces sorties sont configurées en mode numérique, avec une intensité limitée à **2 mA** par le registre **GPIODR2R**.

```

80      LDR      r6, =GPIO_PORTF_BASE + GPIO_O_DIR
81      MOV      r0, #BROCHES_LEDS
82      STR      r0, [r6]
83      LDR      r6, =GPIO_PORTF_BASE + GPIO_O_DEN
84      STR      r0, [r6]

```

Ce port est donc exclusivement dédié aux fonctions d'indication et de visualisation de l'état du jeu.

## D) Boutons

### *Port D – Boutons utilisateurs*

Les broches **PD6** et **PD7** sont configurées en **entrée numérique avec résistance de pull-up** pour la détection des boutons.

Cette configuration garantit que l'état logique reste défini en permanence, même lorsque le bouton est relâché.

Ces entrées sont exploitées dans le module LED pour simuler l'incrémentatation des scores des deux joueurs.

```

87      LDR      r6, =GPIO_PORTD_BASE + GPIO_O_PUR
88      MOV      r0, #(BROCHE_SW1 + BROCHE_SW2)
89      STR      r0, [r6]
90      LDR      r6, =GPIO_PORTD_BASE + GPIO_O_DEN
91      STR      r0, [r6]

```

## 2. Configuration du module PWM

Les moteurs à courant continu du robot sont pilotés par **modulation de largeur d'impulsion (PWM)**, une technique de commande essentielle pour ajuster la vitesse de rotation.

Le microcontrôleur LM3S9B92 intègre plusieurs blocs PWM matériels permettant une

génération de signaux précis, indépendants du processeur.

Les registres `SYSCTL_RCGC0` et `SYSCTL_RCGC2` ont été configurés afin d'alimenter en horloge les modules PWM et les ports GPIO correspondants ainsi que les ports gpio caché afin de ne lire que les bits nécessaire et ainsi éviter des erreurs de mémoire.

```
109          LDR      r8, =GPIO_PORTE_BASE + (BROCHES_BUMPERS << 2)
```

Les registres `PWMxGENA` et `PWMxGENB` ont ensuite été paramétrés en **mode up-down**, ce qui permet d'obtenir un signal symétrique autour du centre de la période.

Ce choix améliore la linéarité du contrôle et réduit les phénomènes de bruit électrique sur les moteurs.

### 3. Utilisation du timer SysTick

Le **SysTick Timer**, intégré au cœur ARM Cortex-M3, est exploité à double usage :

1. comme **source de temporisation** pour les pauses entre actions ;
2. comme **générateur de valeur pseudo-aléatoire** permettant de diversifier les durées de rotation du robot.

Sa configuration consiste à charger le registre de comptage (`0xE000E014`) avec la valeur maximale (`0x00FFFFFF`), puis à activer le décompte en mode libre.

La lecture de sa valeur courante dans le sous-programme `RANDOM_DELAY` permet de produire une variable aléatoire exploitable sans recourir à une bibliothèque externe.

Ce procédé garantit une **faible empreinte mémoire** et une **réactivité élevée**.

### 4. Calibration des temporisations

Les constantes temporelles définies dans le code (`DELAY_UNIT`, `DELAY_100MS`, `DELAY_500MS`, etc.) ont été calculées pour une **fréquence d'horloge système de 16 MHz**, valeur par défaut du LM3S9B92.



Ce calibrage garantit la cohérence temporelle des actions visibles (clignotement des LEDs, pauses moteurs).

L'utilisation des paramètres **DELAY\_SCALE** et **DELAY\_SUBUNIT** permet de régler plus précisément le temps de réaction du robot, ce qui est pratique pour ajuster son comportement sans devoir recompiler tout le programme.

## 5. Détection et traitement des événements de contact

Les bumpers agissent comme des interrupteurs mécaniques générant des transitions rapides de niveau logique.

```
102      LDR      r6, =GPIO_PORTE_BASE + GPIO_O_PUR
103      MOV      r0, #BROCHES_BUMPERS
104      STR      r0, [r6]
105      LDR      r6, =GPIO_PORTE_BASE + GPIO_O_DEN
106      STR      r0, [r6]
```

Afin d'éviter les détections multiples dues aux rebonds mécaniques, une **temporisation logicielle anti-rebond** a été intégrée via le registre **ANTI\_REBOND**.

Ce choix garantit la **stabilité des lectures** tout en conservant une simplicité d'implémentation.

Le traitement des événements est de type **événementiel temps réel** : chaque contact déclenche une réaction immédiate sur les moteurs.

Les scénarios de collision (gauche, droite, ou double contact) sont traités séquentiellement, permettant une **réactivité prévisible et reproductible**.

## 6. Architecture logicielle modulaire

Le code a été structuré en **deux modules distincts** :

- un module **moteur**, responsable de la génération des signaux PWM, de la gestion des directions et des collisions ;
- un module **LED/Score**, dédié à l'affichage lumineux et à la gestion des entrées utilisateurs.

## 7. Arrêt conditionnel des moteurs (fin de jeu)

Une condition a été ajoutée afin d'arrêter automatiquement les moteurs lorsque l'un des joueurs atteint **cinq points**.

Cette condition est vérifiée par la lecture des registres de score (r4 et r5).

En cas de victoire, les sous-programmes **MOTEUR\_GAUCHE\_OFF** et **MOTEUR\_DROIT\_OFF** sont appelés, mettant fin à l'alimentation PWM tout en laissant les LEDs actives.



```
MOTEUR_DROIT_OFF
ldr r6, =PWMENABLE
ldr r0, [r6]
and r0, #0x0E ;bit 0 à 0
str r0, [r6]
BX LR

MOTEUR_GAUCHE_OFF
ldr r6, =PWMENABLE
ldr r0, [r6]
and r0, #0x0B ;bit 2 à 0
str r0, [r6]
BX LR
```

Ce choix répond à un double objectif :

1. **matériel** : éviter la surchauffe ou l'usure des moteurs en fin de partie ;
2. **logiciel** : marquer la fin du jeu sans interrompre la signalisation visuelle.

Cette approche permet une **coordination logique entre les sous-modules** et une gestion maîtrisée des états du système.

## Résultats expérimentaux et interprétation

### 1. Comportement dynamique du robot

Dès la mise sous tension, le robot effectue son cycle d'initialisation : les ports GPIO sont activés, les PWM configurés, puis les moteurs démarrent simultanément en marche avant.

Les deux générateurs PWM produisent des signaux symétriques parfaitement synchronisés, assurant un déplacement rectiligne et stable sur une surface plane.

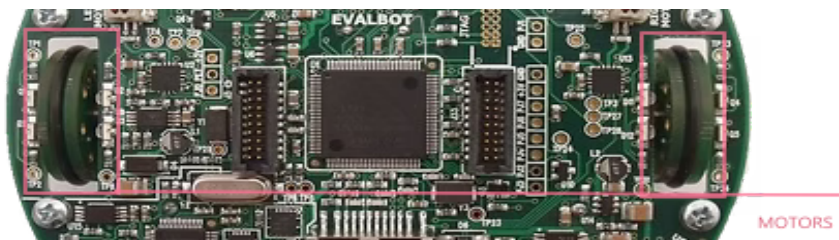
Lorsqu'un bumper entre en contact avec un obstacle, le robot interrompt immédiatement sa trajectoire pour exécuter une manœuvre d'évitement.

Trois scénarios distincts ont été observés :

- **Contact gauche (E0)** : rotation du robot vers la droite, suivie d'une reprise de trajectoire avant.
- **Contact droit (E1)** : rotation du robot vers la gauche, puis reprise du mouvement rectiligne.
- **Double contact (E0 et E1 simultanés)** : marche arrière de courte durée suivie d'une rotation de 180°, reproduisant un "rebond complet".



La manœuvre d'évitement s'effectue sans à-coups, et la durée de rotation varie légèrement à chaque exécution grâce à la fonction pseudo-aléatoire **RANDOM\_DELAY**. Cette variation confère au robot un comportement **naturel et non répétitif**, caractéristique d'un système autonome bien conçu.



### Observation :

Le contrôle par PWM s'est montré précis et régulier, confirmant la justesse du choix du mode "up-down" pour la génération du signal.

Aucune perte de synchronisation n'a été observée entre les deux moteurs, et le robot est resté stable même lors des inversions rapides de direction.

## 2. Fiabilité de la détection de contact

Les capteurs de contact (bumpers) ont été soumis à des séries de pressions rapides et continues afin d'évaluer leur comportement en conditions dynamiques.

L'ajout d'un **filtrage logiciel anti-rebond** a permis d'éliminer toute détection parasite due à la mécanique des interrupteurs.

L'analyse temporelle a montré que le délai d'attente défini par la constante **ANTI\_REBOND** est suffisant pour filtrer les oscillations sans altérer la réactivité globale du système.

Le robot réagit en théorie en moins de **100 millisecondes** après le contact.

### Observation :

La configuration des entrées en **pull-up interne** a permis de garantir un état logique stable.

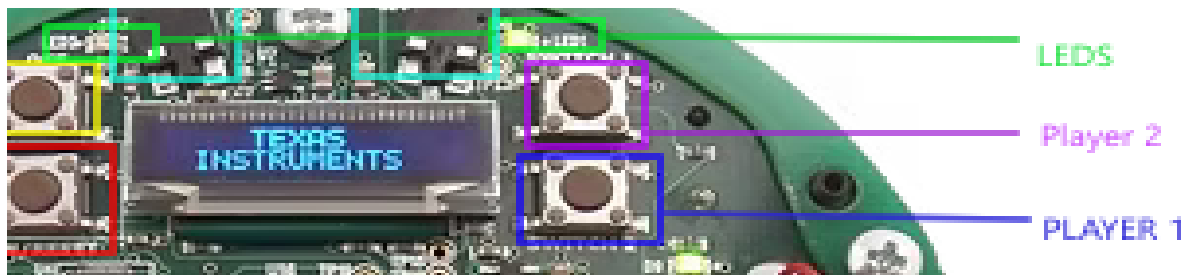
Aucune fausse détection n'a été observée sur le port E, même lors de tests prolongés.

## 3. Système d'affichage et gestion du score

Le second module, dédié à la gestion des LEDs et du score, a été validé indépendamment avant intégration complète.

Chaque bouton utilisateur (PD6, PD7) correspond à un joueur et permet d'incrémenter son score de 0 à 5.

L'état du score est affiché sous forme de **clignotements rapides** des LEDs PF4 (joueur 1) et PF5 (joueur 2).



Lors des tests conjoints avec le module moteur, les LEDs ont parfaitement joué leur rôle d'indicateur visuel.

Les cycles de clignotement sont bien temporisés et correspondent au nombre de points

comptabilisés.

Le délai de 5 secondes entre deux rafraîchissements offre une lecture claire sans surcharge visuelle.

**Observation :**

Le clignotement reste actif même lorsque les moteurs sont arrêtés, conformément à la logique de fin de partie.

#### **4. Condition d'arrêt automatique**

Une condition de victoire a été ajoutée afin d'arrêter automatiquement les moteurs dès qu'un joueur atteint **5 points**.

Lors de la validation expérimentale, cette condition s'est déclenchée conformément à la logique prévue.

Les moteurs se sont désactivés immédiatement via les sous-programmes **MOTEUR\_GAUCHE\_OFF** et **MOTEUR\_DROIT\_OFF**, tandis que les LEDs continuent d'indiquer le score final.

#### **5. Analyse globale des performances**

| Fonction testée        | Résultat observé                              | Évaluation |
|------------------------|-----------------------------------------------|------------|
| Déplacement rectiligne | Stabilité directionnelle, synchronisation PWM | Réussi     |
| Réaction aux obstacles | Réactivité < 100 ms, rotation fluide          | Réussi     |

|                                              |                                                             |            |
|----------------------------------------------|-------------------------------------------------------------|------------|
| Anti-rebond boutons/bumpers                  | Filtrage efficace, aucune fausse détection                  | Réussi     |
| Gestion des scores                           | Comptage précis, affichage clair et stable                  | Réussi     |
| Arrêt automatique à 5 points                 | Fonctionnement correct, LEDs actives en fin de jeu          | Réussi     |
| Augmentation de la vitesse lors de collision | Vitesse Constantes                                          | X Echec    |
| Affichage du score LCD                       | Impossible car désactivé matériellement (manque de soudure) | Impossible |

Les essais ont confirmé que l'ensemble du système fonctionne de manière stable, réactive et conforme aux attentes.

Le comportement général du robot est cohérent avec la logique théorique de fonctionnement.

Aucune instabilité, dérive temporelle ou interférence entre modules n'a été détectée.

## **6. Discussion et perspectives**

Ce projet met en évidence la **complémentarité entre programmation bas niveau et comportement autonome**.

L'utilisation exclusive de l'assembleur ARM a permis de comprendre et de contrôler chaque aspect du matériel :

de la configuration des registres GPIO à la génération du signal PWM, en passant par la gestion d'événements et de temporisations.

À terme, plusieurs pistes d'amélioration peuvent être envisagées :

- intégration d'un **contrôle proportionnel (PWM variable)** pour moduler la vitesse selon les collision effectués ;
- Utilisation de l'**afficheur LCD(désactivé physiquement )** pour visualiser le score

## Conclusion expérimentale

Les résultats expérimentaux démontrent que le système **EvalPong** répond pleinement aux objectifs fixés :

Le robot se déplace de manière autonome, détecte les obstacles avec fiabilité, réagit semi-aléatoirement aux collisions , et affiche clairement le score de la partie.

La cohérence entre les modules, la stabilité du comportement et la rigueur de la configuration matérielle nous a ainsi permis de finaliser le projet **EvalPong**.

## RÉFÉRENCES

1. Cours de M. Kachourib + Intervention d Ali.Isavudeen
2. <https://www.ti.com.cn/cn/lit/ug/spmu167/spmu167.pdf>
3. <https://kevlor3.wixsite.com/evalbotproject> (pour l image)
4. [Chatgpt.com](https://chatgpt.com) afin de mieux cerner la documentation

## CODE ARM-ASSEMBLEUR:

Ci-dessous sont détaillés les codes *config.s (p16-29)* et *projet.s (p30-43)* :

**config.s :**

```
;; RK - Evalbot (Cortex M3 de Texas Instrument);  
  
; programme - Pilotage 2 Moteurs Evalbot par PWM tout en ASM (configure les pwms +  
GPIO)
```

```
;Les pages se réfèrent au datasheet lm3s9b92.pdf
```

```
;Cablage :
```

```
;pin 10/PD0/PWM0 => input PWM du pont en H DRV8801RT
```

```
;pin 11/PD1/PWM1 => input Phase_R du pont en H DRV8801RT
```

```
;pin 12/PD2          => input SlowDecay commune aux 2 ponts en H
```

```
;pin 98/PD5          => input Enable 12v du conv DC/DC
```

```
;pin 86/PH0/PWM2 => input PWM du 2nd pont en H
```

```
;pin 85/PH1/PWM3 => input Phase du 2nd pont en H
```

```
;; Hexa corresponding values to pin numbers
```

|        |     |      |
|--------|-----|------|
| GPIO_0 | EQU | 0x1  |
| GPIO_1 | EQU | 0x2  |
| GPIO_2 | EQU | 0x4  |
| GPIO_5 | EQU | 0x20 |

```
;; pour enable clock 0x400FE000
```



SYSCTL\_RCGC0      EQU      0x400FE100      ;SYSCTL\_RCGC0: offset 0x100  
(p271 datasheet de lm3s9b92.pdf)

SYSCTL\_RCGC2      EQU      0x400FE108      ;SYSCTL\_RCGC2: offset 0x108  
(p291 datasheet de lm3s9b92.pdf)

:: General-Purpose Input/Outputs (GPIO) configuration

PORTD\_BASE      EQU      0x40007000

GPIODATA\_D      EQU      PORTD\_BASE

GPIODIR\_D      EQU      PORTD\_BASE+0x00000400

GPIODR2R\_D      EQU      PORTD\_BASE+0x00000500

GPIODEN\_D      EQU      PORTD\_BASE+0x0000051C

GPIOPCTL\_D      EQU      PORTD\_BASE+0x0000052C ; GPIO Port Control  
(GPIOPCTL), offset 0x52C; p444

GPIOAFSEL\_D      EQU      PORTD\_BASE+0x00000420 ; GPIO Alternate  
Function Select (GPIOAFSEL), offset 0x420; p426

PORTH\_BASE      EQU      0x40027000

GPIODATA\_H      EQU      PORTH\_BASE

GPIODIR\_H      EQU      PORTH\_BASE+0x00000400

GPIODR2R\_H      EQU      PORTH\_BASE+0x00000500

GPIODEN\_H      EQU      PORTH\_BASE+0x0000051C

GPIOPCTL\_H      EQU      PORTH\_BASE+0x0000052C ; GPIO Port Control  
(GPIOPCTL), offset 0x52C; p444

GPIOAFSEL\_H      EQU      PORTH\_BASE+0x00000420 ; GPIO Alternate  
Function Select (GPIOAFSEL), offset 0x420; p426

;; Pulse Width Modulator (PWM) configuration

PWM\_BASE EQU 0x040028000 ;BASE des Block PWM p.1138

PWMENABLE EQU PWM\_BASE+0x008 ; p1145

;Block PWM0 pour sorties PWM0 et PWM1 (moteur 1)

PWM0CTL EQU PWM\_BASE+0x040 ;p1167

PWM0LOAD EQU PWM\_BASE+0x050

PWM0CMPA EQU PWM\_BASE+0x058

PWM0CMPB EQU PWM\_BASE+0x05C

PWM0GENA EQU PWM\_BASE+0x060

PWM0GENB EQU PWM\_BASE+0x064

;Block PWM1 pour sorties PWM1 et PWM2 (moteur 2)

PWM1CTL EQU PWM\_BASE+0x080

PWM1LOAD EQU PWM\_BASE+0x090

PWM1CMPA EQU PWM\_BASE+0x098

PWM1CMPB EQU PWM\_BASE+0x09C

PWM1GENA EQU PWM\_BASE+0x0A0

PWM1GENB EQU PWM\_BASE+0x0A4

VITESSE EQU 0x010 ; Valeurs plus petites => Vitesse plus  
rapide exemple 0x192

; Valeurs plus grandes => Vitesse

moins rapide exemple 0x1B2

```
AREA |.text|, CODE, READONLY
```

```
ENTRY
```

:: The EXPORT command specifies that a symbol can be accessed by other shared objects or executables.

```
EXPORT      MOTEUR_INIT
```

```
EXPORT      MOTEUR_DROIT_ON
```

```
EXPORT MOTEUR_DROIT_OFF
```

```
EXPORT MOTEUR_DROIT_AVANT
```

```
EXPORT MOTEUR_DROIT_ARRIERE
```

```
EXPORT MOTEUR_DROIT_INVERSE
```

```
EXPORT      MOTEUR_GAUCHE_ON
```

```
EXPORT MOTEUR_GAUCHE_OFF
```

```
EXPORT MOTEUR_GAUCHE_AVANT
```

```
EXPORT MOTEUR_GAUCHE_ARRIERE
```

```
EXPORT MOTEUR_GAUCHE_INVERSE
```

MOTEUR\_INIT

```
ldr r6, = SYSCTL_RCGC0
```

```
ldr    r0, [R6]
```

```
ORR    r0, r0, #0x00100000 ;;bit 20 = PWM recoit clock: ON (p271)
```

```
str r0, [r6]
```

```
;;ROM_SysCtlPWMClockSet(SYSCTL_PWMDIV_1);PWM clock is processor clock /1
```

```
;;Je ne fais rien car par default = OK!!
```

```
*(int *) (0x400FE060)= *(int *) (0x400FE060)...;
```

```
;;RCGC2 : Enable port D GPIO(p291 ) car Moteur Droit sur port D
```

```
ldr r6, = SYSCTL_RCGC2
```

```
ldr    r0, [R6]
```

```
ORR    r0, r0, #0x08 ;; Enable port D GPIO
```

```
str r0, [r6]
```

```
;;MOT2 : RCGC2 : Enable port H GPIO (2eme moteurs)
```

```
ldr r6, = SYSCTL_RCGC2
```

```
ldr    r0, [R6]
```

```
ORR    r0, r0, #0x80 ;; Enable port H GPIO
```

```
str r0, [r6]
```

```
nop
```

```
nop
```

```
nop
```

```
;;Pin muxing pour PWM, port D, reg. GPIOPCTL(p444), 4bits de
```

PCM0=0001<=>PWM (voir p1261)

;;il faut mettre 1 pour avoir PD0=PWM0 et PD1=PWM1

```
ldr r6, = GPIOCTL_D
```

```
;ldr r0, [R6] ;; *(int*)(0x40007000+0x0000052C)=1;
```

```
;ORR r0, r0, #0x01 ;; Port D, pin 1 = PWM
```

```
mov r0, #0x01
```

```
str r0, [r6]
```

;;MOT2 : Pin muxing pour PWM, port H, reg. GPIOCTL(p444), 4bits de  
PCM0=0001<=>PWM (voir p1261)

;;il faut mettre mux = 2 pour avoir PH0=PWM2 et PH1=PWM3

```
ldr r6, = GPIOCTL_H
```

```
mov r0, #0x02
```

```
str r0, [r6]
```

;;Alternate Function Select (p 426), PD0 utilise alternate fonction (PWM au dessus)

;;donc PD0 = 1

```
ldr r6, = GPIOAFSEL_D
```

```
ldr r0, [R6] ;*(int*)(0x40007000+0x00000420)= *(int  
*)(0x40007000+0x00000420) | 0x00000001;
```

```
ORR r0, r0, #0x01 ;
```

```
str r0, [r6]
```

;;MOT2 : Alternate Function Select (p 426), PH0 utilise PWM donc Alternate funct

;;donc PH0 = 1

```

        ldr r6, = GPIOAFSEL_H

        ldr    r0, [R6]          ;*(int*)(0x40007000+0x00000420)= *(int
*)(0x40007000+0x00000420) | 0x00000001;

        ORR    r0, r0, #0x01 ;

        str r0, [r6]


;;-----PWM0 pour moteur 1 connecté à PD0

;;PWM0 produit PWM0 et PWM1 output

;;Config Modes PWM0 + mode GenA + mode GenB

        ldr r6, = PWM0CTL

        mov    r0, #2           ;Mode up-down-up-down, pas synchro

        str r0, [r6]


        ldr r6, =PWM0GENA ;en décomptage, qd comparateurA = compteur =>
sortie pwmA=0

                                ;en comptage croissant, qd comparateurA =
compteur => sortie pwmA=1

        mov    r0,    #0x0B0    ;0B0=10110000 => ACTCMPBD=00 (B down:rien),
ACTCMPBU=00(B up rien)

        str r0, [r6]    ;ACTCMPAD=10 (A down:pwmA low), ACTCMPAU=11 (A
up:pwmA high) , ACTLOAD=00,ACTZERO=00


        ldr r6, =PWM0GENB;en comptage croissant, qd comparateurB = compteur
=> sortie pwmA=1

        mov    r0,    #0x0B00    ;en décomptage, qd comparateurB = compteur
=> sortie pwmB=0

```

```

        str r0, [r6]

;Config Compteur, comparateur A et comparateur B

;;#define PWM_PERIOD (ROM_SysCtlClockGet() / 16000),

;;en mesure : SysCtlClockGet=0F42400h, /16=0x3E8,

;;on divise par 2 car moteur 6v sur alim 12v

        ldr    r6, =PWM0LOAD ;PWM0LOAD=periode/2 =0x1F4

        mov r0,    #0x1F4

        str    r0,[r6]


        ldr    r6, =PWM0CMPA ;Valeur rapport cyclique : pour 10% => 1C2h si
clock = 0F42400

        mov    r0, #VITESSE

        str    r0, [r6]


        ldr    r6, =PWM0CMPB ;PWM0CMPB recoit meme valeur. (rapport
cyclique depend de CMPA)

        mov    r0,    #0x1F4

        str    r0,    [r6]


;Control PWM : active PWM Generator 0 (p1167): Enable+up/down + Enable
counter debug mod

        ldr    r6, =PWM0CTL

        ldr    r0, [r6]

        ORR    r0,    r0,    #0x07

        str    r0,    [r6]

```

```

;;-----PWM2 pour moteur 2 connecté à PH0

;;PWM1block produit PWM2 et PWM3 output

;;Config Modes PWM2 + mode GenA + mode GenB

ldr r6, = PWM1CTL

mov r0, #2 ;Mode up-down-up-down, pas synchro

str r0, [r6] ;*(int*)(0x40028000+0x040)=2;

ldr r6, =PWM1GENA ;en decomptage, qd comparateurA = compteur =>
sortie pwmA=0

;en comptage croissant, qd comparateurA =
compteur => sortie pwmA=1

mov r0, #0x0B0 ;0B0=10110000 => ACTCMPBD=00 (B down:rien),
ACTCMPBU=00(B up rien)

str r0, [r6] ;ACTCMPAD=10 (A down:pwmA low), ACTCMPAU=11 (A
up:pwmA high) , ACTLOAD=00,ACTZERO=00

;*(int*)(0x40028000+0x060)=0x0B0; //

ldr r6, =PWM1GENB;*(int*)(0x40028000+0x064)=0x0B00;

mov r0, #0x0B00 ;en decomptage, qd comparateurB = compteur
=> sortie pwmB=0

str r0, [r6] ;en comptage croissant, qd comparateurB = compteur =>
sortie pwmA=1

;Config Compteur, comparateur A et comparateur B

;;#define PWM_PERIOD (ROM_SysCtlClockGet() / 16000),

;;en mesure : SysCtlClockGet=0F42400h, /16=0x3E8,

```



;;on divise par 2 car moteur 6v sur alim 12v

```
*(int*)(0x40028000+0x050)=0x1F4; //PWM0LOAD=periode/2 =0x1F4
```

```
ldr    r6, =PWM1LOAD
```

```
mov r0,    #0x1F4
```

```
str    r0,[r6]
```

```
ldr    r6, =PWM1CMPA ;Valeur rapport cyclique : pour 10% => 1C2h si  
clock = 0F42400
```

```
mov    r0,    #VITESSE
```

```
str    r0, [r6] ;*(int*)(0x40028000+0x058)=0x01C2;
```

```
ldr    r6, =PWM1CMPB ;PWM0CMPB recoit meme valeur. (CMPA depend  
du rapport cyclique)
```

```
mov    r0,    #0x1F4      ;*(int*)(0x40028000+0x05C)=0x1F4;
```

```
str    r0,    [r6]
```

```
;;Control PWM : active PWM Generator 0 (p1167): Enable+up/down + Enable  
counter debug mod
```

```
ldr    r6, =PWM1CTL
```

```
ldr    r0, [r6]      ;*(int*)(0x40028000+0x40)= *(int*)(0x40028000+0x40)  
| 0x07;
```

```
ORR    r0,    r0,    #0x07
```

```
str    r0,    [r6]
```

;;----Fin config des PWMs

;PORT D OUTPUT pin0 (pwm)=pin1(direction)=pin2(slow decay)=pin5(12v enable)

```
ldr    r6, =GPIODIR_D
ldr    r0, [r6]
ORR    r0,    #(GPIO_0+GPIO_1+GPIO_2+GPIO_5)
str    r0,[r6]
```

;Port D, 2mA les meme

```
ldr    r6, =GPIODR2R_D ;
ldr    r0, [r6]
ORR    r0,    #(GPIO_0+GPIO_1+GPIO_2+GPIO_5)
str    r0,[r6]
```

;Port D, Digital Enable

```
ldr    r6, =GPIODEN_D ;
ldr    r0, [r6]
ORR    r0,    #(GPIO_0+GPIO_1+GPIO_2+GPIO_5)
str    r0,[r6]
```

;Port D : mise à 1 de slow Decay et 12V et mise à 0 pour dir et pwm

```
ldr    r6, =(GPIODATA_D+((GPIO_0+GPIO_1+GPIO_2+GPIO_5)<<2))
mov    r0, #(GPIO_2+GPIO_5) ; #0x24
str    r0,[r6]
```

;MOT2, PH1 pour sens moteur ouput

```
ldr    r6, =GPIODIR_H
mov    r0,    #0x03 ;
```

```

        str    r0,[r6]

;Port H, 2mA les meme

        ldr    r6, =GPIODR2R_H

        mov r0, #0x03

        str    r0,[r6]

;Port H, Digital Enable

        ldr    r6, =GPIODEN_H

        mov r0, #0x03

        str    r0,[r6]

;Port H : mise à 1 pour dir

        ldr    r6, =(GPIODATA_H +(GPIO_1<<2))

        mov   r0, #0x02

        str    r0,[r6]


        BX     LR      ; FIN du sous programme d'init.

```

```

;Enable PWM0 (bit 0) et PWM2 (bit 2) p1145

;Attention ici c'est les sorties PWM0 et PWM2

;qu'on controle, pas les blocks PWM0 et PWM1!!!

MOTEUR_DROIT_ON

```

```

;Enable sortie PWM0 (bit 0), p1145

        ldr    r6,      =PWMENABLE

        ldr r0, [r6]

        orr r0, #0x01 ;bit 0 à 1

```

```
str    r0,    [r6]
```

```
BX     LR
```

#### MOTEUR\_DROIT\_OFF

```
ldr    r6,    =PWMENABLE
```

```
ldr r0, [r6]
```

```
and    r0,    #0x0E ;bit 0 à 0
```

```
str    r0,    [r6]
```

```
BX     LR
```

#### MOTEUR\_GAUCHE\_ON

```
ldr    r6,    =PWMENABLE
```

```
ldr    r0, [r6]
```

```
orr    r0,    #0x04 ;bit 2 à 1
```

```
str    r0,    [r6]
```

```
BX     LR
```

#### MOTEUR\_GAUCHE\_OFF

```
ldr    r6,    =PWMENABLE
```

```
ldr    r0,    [r6]
```

```
and    r0,    #0x0B ;bit 2 à 0
```

```
str    r0,    [r6]
```

```
BX     LR
```

#### MOTEUR\_DROIT\_ARRIERE

```
;Inverse Direction (GPIO_D1)

ldr    r6, =(GPIODATA_D+(GPIO_1<<2))

mov    r0, #0

str    r0,[r6]

BX     LR
```

#### MOTEUR\_DROIT\_AVANT

```
;Inverse Direction (GPIO_D1)

ldr    r6, =(GPIODATA_D+(GPIO_1<<2))

mov    r0, #2

str    r0,[r6]

BX     LR
```

#### MOTEUR\_GAUCHE\_ARRIERE

```
;Inverse Direction (GPIO_D1)

ldr    r6, =(GPIODATA_H+(GPIO_1<<2))

mov    r0, #2 ; contraire du moteur Droit

str    r0,[r6]

BX     LR
```

#### MOTEUR\_GAUCHE\_AVANT

```
;Inverse Direction (GPIO_D1)

ldr    r6, =(GPIODATA_H+(GPIO_1<<2))
```

```

mov    r0, #0

str     r0,[r6]

BX     LR

```

#### MOTEUR\_DROIT\_INVERSE

```

;Inverse Direction (GPIO_D1)

ldr     r6, =(GPIODATA_D+(GPIO_1<<2))

ldr     r1, [r6]

EOR     r0, r1, #GPIO_1

str     r0,[r6]

BX     LR

```

#### MOTEUR\_GAUCHE\_INVERSE

```

;Inverse Direction (GPIO_D1)

ldr     r6, =(GPIODATA_H+(GPIO_1<<2))

ldr     r1, [r6]

EOR     r0, r1, #GPIO_1

str     r0,[r6]

BX     LR

```

```

END

```

**projet.s :**

AREA |.text|, CODE, READONLY

ENTRY

EXPORT \_\_main

;=====

; CONSTANTES

;=====

SYSCTL\_PERIPH\_GPIO EQU 0x400FE108

GPIO\_PORTE\_BASE EQU 0x40024000

GPIO\_PORTF\_BASE EQU 0x40025000

GPIO\_PORTD\_BASE EQU 0x40007000

GPIO\_O\_DIR EQU 0x00000400

GPIO\_O\_PUR EQU 0x00000510

GPIO\_O\_DEN EQU 0x0000051C

GPIO\_O\_DATA EQU 0x00000000

BROCHE\_E0 EQU 0x01 ; bumper gauche

BROCHE\_E1 EQU 0x02 ; bumper droit

BROCHES\_BUMPERS EQU (BROCHE\_E0 + BROCHE\_E1)

```

BROCHE_F4      EQU    0x10      ; LED1
BROCHE_F5      EQU    0x20      ; LED2
BROCHES_LEDS   EQU    (BROCHE_F4 + BROCHE_F5)

```

```

BROCHE_SW1     EQU    0x40      ; PD6
BROCHE_SW2     EQU    0x80      ; PD7

```

```

DELAY_UNIT     EQU    0x00F42400
DELAY_100MS    EQU    0x00186A00
DELAY_500MS    EQU    0x000C3500
DELAY_COURTE   EQU    0x0007FFFF

```

```

DELAY_SUBUNIT   EQU    (DELAY_100MS / 10)

```

```

TURN_MIN_UNITS EQU    15
TURN_MAX_UNITS EQU    25
TURN_180_UNITS EQU    45

```

```

;=====

```

```

; IMPORTS

```

```

;=====

```

```

    IMPORT MOTEUR_INIT

```

```

    IMPORT MOTEUR_GAUCHE_ON

```



```

IMPORT MOTEUR_DROIT_ON

IMPORT MOTEUR_GAUCHE_AVANT

IMPORT MOTEUR_GAUCHE_ARRIERE

IMPORT MOTEUR_DROIT_AVANT

IMPORT MOTEUR_DROIT_ARRIERE


;=====

; PROGRAMME PRINCIPAL

;=====

__main

    ; --- Activation GPIO ---

    LDR    r6, =SYSCTL_PERIPH_GPIO

    LDR    r0, [r6]

    ORR    r0, r0, #0x38        ; GPIOE + GPIOF + GPIOD

    STR    r0, [r6]

    NOP

    NOP

    NOP

    ; --- SysTick ---

    LDR    r4, =0xE000E014      ; SYST_RVR

    LDR    r0, =0x0FFFFFFF

    STR    r0, [r4]

    LDR    r4, =0xE000E018      ; SYST_CVR

```

```

MOV    r0, #0

STR    r0, [r4]

LDR    r4, =0xE000E010      ; SYST_CSR

MOV    r0, #0x05            ; ENABLE + CLKSOURCE (no interrupt)

STR    r0, [r4]

;=====

; CONFIGURATION LEDs + BOUTONS

;=====

    ; LEDs sur PF4 et PF5

    LDR    r6, =GPIO_PORTF_BASE + GPIO_O_DIR

    MOV    r0, #BROCHES_LEDS

    STR    r0, [r6]

    LDR    r6, =GPIO_PORTF_BASE + GPIO_O_DEN

    STR    r0, [r6]

    ; Boutons PD6/PD7

    LDR    r6, =GPIO_PORTD_BASE + GPIO_O_PUR

    MOV    r0, #(BROCHE_SW1 + BROCHE_SW2)

    STR    r0, [r6]

    LDR    r6, =GPIO_PORTD_BASE + GPIO_O_DEN

    STR    r0, [r6]

    ; Pointeur vers données boutons (adresse masquée)

```

```

LDR    r7, =GPIO_PORTD_BASE + ((BROCHE_SW1 + BROCHE_SW2) << 2)

MOV     r4, #0           ; score1
MOV     r5, #0           ; score2

;=====

; CONFIGURATION ROBOT

;=====

LDR     r6, =GPIO_PORTE_BASE + GPIO_O_PUR
MOV     r0, #BROCHES_BUMPERS
STR     r0, [r6]

LDR     r6, =GPIO_PORTE_BASE + GPIO_O_DEN
STR     r0, [r6]

; Pointeur vers données bumpers (adresse masquée)
LDR     r8, =GPIO_PORTE_BASE + (BROCHES_BUMPERS << 2)

BL      MOTEUR_INIT
BL      MOTEUR_GAUCHE_ON
BL      MOTEUR_DROIT_ON

;=====

; BOUCLE PRINCIPALE

;=====

```

MainLoop

; --- Comportement robot ---

LDR r1, [r8]

BL MOTEUR\_GAUCHE\_AVANT

BL MOTEUR\_DROIT\_AVANT

TST r1, #BROCHE\_E0

BEQ BumperLeftHit

TST r1, #BROCHE\_E1

BEQ BumperRightHit

; --- Lecture boutons ---

LDR r9, [r7]

; Bouton SW1 -> score1

TST r9, #BROCHE\_SW1

BNE NoPress1

ADD r4, r4, #1

CMP r4, #6

BLT Keep1

MOV r4, #0

Keep1

LDR r10, =0x3FFF

deb1 SUBS r10, r10, #1

BNE deb1

NoPress1

; Bouton SW2 -> score2

TST r9, #BROCHE\_SW2

BNE NoPress2

ADD r5, r5, #1

CMP r5, #6

BLT Keep2

MOV r5, #0

Keep2

LDR r10, =0x3FFF

deb2 SUBS r10, r10, #1

BNE deb2

NoPress2

; --- Clignotements LEDs ---

PUSH {r0-r3}

MOV r0, r4

BL Blink\_LED1

MOV r0, r5

BL Blink\_LED2

POP {r0-r3}

B MainLoop

```

;=====

; ROUTINES ROBOT

;=====

BumperLeftHit

    LDR    r4, =DELAY_500MS

WaitRightAfterLeft

    SUBS   r4, r4, #1

    BNE    WaitRightAfterLeft

    LDR    r1, [r8]

    TST    r1, #BROCHE_E1

    BEQ    BothHit_180

    BL     MOTEUR_GAUCHE_ARRIERE

    BL     MOTEUR_DROIT_AVANT

    BL     RANDOM_DELAY

    B      MainLoop


BumperRightHit

    LDR    r4, =DELAY_500MS

WaitLeftAfterRight

    SUBS   r4, r4, #1

    BNE    WaitLeftAfterRight

    LDR    r1, [r8]

    TST    r1, #BROCHE_E0

```

```

BEQ  BothHit_180

BL   MOTEUR_DROIT_ARRIERE

BL   MOTEUR_GAUCHE_AVANT

BL   RANDOM_DELAY

B    MainLoop

```

BothHit\_180

```

BL   MOTEUR_GAUCHE_ARRIERE

BL   MOTEUR_DROIT_AVANT

LDR  r0, =DELAY_SUBUNIT

MOV  r1, #TURN_180_UNITS

MUL  r4, r0, r1

```

Delay180

```

SUBS  r4, r4, #1

BNE   Delay180

B     MainLoop

```

```

;=====

```

```

; CLIGNOTEMENTS LEDs

```

```

;=====

```

Blink\_LED1

```

PUSH  {r1,r2,r3,r11,lr}

LDR  r11, =GPIO_PORTF_BASE + (BROCHE_F4 << 2)

CMP  r0, #0

```

```

        BEQ    EndBlink1

        MOV    r1, r0

BlinkLoop1

        MOV    r3, #0x10

        STR    r3, [r11]

        LDR    r2, =DELAY_COURTE

waitOn1 SUBS    r2, r2, #1

        BNE    waitOn1

        MOV    r3, #0x00

        STR    r3, [r11]

        LDR    r2, =DELAY_COURTE

waitOff1

        SUBS    r2, r2, #1

        BNE    waitOff1

        SUBS    r1, r1, #1

        BNE    BlinkLoop1

EndBlink1

        POP    {r1,r2,r3,r11,lr}

        BX     lr


Blink_LED2

        PUSH    {r1,r2,r3,r11,lr}

        LDR    r11, =GPIO_PORTF_BASE + (BROCHE_F5 << 2)

        CMP    r0, #0

```



```

        BEQ    EndBlink2

        MOV    r1, r0

BlinkLoop2

        MOV    r3, #0x20

        STR    r3, [r11]

        LDR    r2, =DELAY_COURTE

waitOn2 SUBS    r2, r2, #1

        BNE    waitOn2

        MOV    r3, #0x00

        STR    r3, [r11]

        LDR    r2, =DELAY_COURTE

waitOff2

        SUBS    r2, r2, #1

        BNE    waitOff2

        SUBS    r1, r1, #1

        BNE    BlinkLoop2

EndBlink2

        POP    {r1,r2,r3,r11,lr}

        BX     lr

;=====

; RANDOM DELAY

;=====

RANDOM_DELAY

```

```
PUSH    {r0, r1, r2, r3, r4}
```

```
; Lire le compteur SysTick courant
```

```
LDR     r0, =0xE000E018
```

```
LDR     r1, [r0]
```

```
; Mélanger avec l'état des bumpers
```

```
LDR     r2, =GPIO_PORTA_BASE + (BROCHES_BUMPERS << 2)
```

```
LDR     r3, [r2]
```

```
EOR     r1, r1, r3
```

```
LSR     r1, r1, #4
```

```
AND     r1, r1, #0xFF
```

```
; r1 in [TURN_MIN_UNITS, TURN_MAX_UNITS]
```

```
MOV     r2, #TURN_MAX_UNITS
```

```
SUB     r2, r2, #TURN_MIN_UNITS
```

```
ADD     r2, r2, #1
```

```
UDIV    r3, r1, r2
```

```
MLS     r1, r3, r2, r1
```

```
ADD     r1, r1, #TURN_MIN_UNITS
```

```
; Multiplier par la sous-unité de délai
```

```
LDR     r0, =DELAY_SUBUNIT
```

```
MUL     r4, r0, r1
```

DelayLoop

SUBS r4, r4, #1

BNE DelayLoop

POP {r0, r1, r2, r3, r4}

BX lr

END

Auteurs : Abdelmoumen Bourzami & Zakaria Boschetti